

AtliQ Hotels Data Analysis Project

Datasets

We have 5 csv file

- dim_date.csv
- dim_hotels.csv
- dim_rooms.csv
- fact_aggregated_bookings
- fact_bookings.csv

In [7]: `import pandas as pd`

In [208... `df_bookings=pd.read_csv(r"D:\Python_study\Python_Practice\source-code\python_projects\3_projects\fact_bookings.csv")`
`df_bookings.head(3)`

Out[208...

	booking_id	property_id	booking_date	check_in_date	checkout_date	no_guests	room_category
0	May012216558RT11	16558	27-04-22	1/5/2022	2/5/2022	-3.0	
1	May012216558RT12	16558	30-04-22	1/5/2022	2/5/2022	2.0	
2	May012216558RT13	16558	28-04-22	1/5/2022	4/5/2022	2.0	

Loading & Exploration of Data for each file

In [9]: `# checking no of rows & columns in that fact table`
`df_bookings.shape`

Out[9]: (134592, 12)

In [176... `# to check if df has any duplicated rows`
`duplicated_rows=df_bookings.duplicated()`
`print(duplicated_rows.sum())`

2

In [209... `# to see the duplicated rows from the data frame`
`df_bookings[df_bookings.duplicated()]`

Out[209...

	booking_id	property_id	booking_date	check_in_date	checkout_date	no_guests	room_category
7	May012216558RT17	16558	28-04-22	1/5/2022	6/5/2022	2.0	
13	May012216558RT112	16558	26-04-22	1/5/2022	2/5/2022	2.0	

In [210... `# to drop duplicated rows from the data frame`
`df_bookings=df_bookings.drop_duplicates()`
`df_bookings.shape`

Out[210... (134590, 12)

```
In [211... # Check how many missing values are in each column
df_bookings.isna().sum()
# we found 77907 null values for ratings_given, which is quite common in hotel industry, because
```

```
Out[211... booking_id          0
property_id         0
booking_date        0
check_in_date       0
checkout_date       0
no_guests           3
room_category       0
booking_platform    0
ratings_given      77907
booking_status      0
revenue_generated   0
revenue_realized    0
dtype: int64
```

```
In [212... #Get rows that contain missing values
df_bookings[df_bookings.isna().any(axis=1)]
```

Out[212...

	booking_id	property_id	booking_date	check_in_date	checkout_date	no_guests	room
1	May012216558RT12	16558	30-04-22	1/5/2022	2/5/2022	2.0	
3	May012216558RT14	16558	28-04-22	1/5/2022	2/5/2022	-2.0	
6	May012216558RT17	16558	28-04-22	1/5/2022	6/5/2022	2.0	
8	May012216558RT18	16558	26-04-22	1/5/2022	3/5/2022	2.0	
9	May012216558RT19	16558	30-04-22	1/5/2022	2/5/2022	2.0	
...	...	...	...	...	...	...	...
134581	Jul312217564RT311	17564	30-07-22	31-07-22	2/8/2022	2.0	
134582	Jul312217564RT41	17564	31-07-22	31-07-22	2/8/2022	1.0	
134585	Jul312217564RT44	17564	31-07-22	31-07-22	1/8/2022	1.0	
134589	Jul312217564RT48	17564	30-07-22	31-07-22	2/8/2022	1.0	
134591	Jul312217564RT410	17564	31-07-22	31-07-22	1/8/2022	2.0	

77908 rows × 12 columns



```
In [213... #checking basic statistics
df_bookings.describe()
```

Out[213...

	property_id	no_guests	ratings_given	revenue_generated	revenue_realized
count	134590.000000	134587.000000	56683.000000	1.345900e+05	134590.000000
mean	18061.113493	2.036170	3.619004	1.537805e+04	12696.123256
std	1093.055847	1.034885	1.235009	9.303604e+04	6928.108124
min	16558.000000	-17.000000	1.000000	6.500000e+03	2600.000000
25%	17558.000000	1.000000	3.000000	9.900000e+03	7600.000000
50%	17564.000000	2.000000	4.000000	1.350000e+04	11700.000000
75%	18563.000000	2.000000	5.000000	1.800000e+04	15300.000000
max	19563.000000	6.000000	5.000000	2.856000e+07	45220.000000

In [214...

```
# check out max & min revenue_generated per booking
df_bookings['revenue_generated'].max(),df_bookings['revenue_generated'].min()
```

Out[214...

```
(np.int64(28560000), np.int64(6500))
```

In [18]:

```
# check unique room_category
df_bookings['room_category'].unique()
```

Out[18]:

```
array(['RT1', 'RT2', 'RT3', 'RT4'], dtype=object)
```

In [19]:

```
# check unique booking_platform
df_bookings['booking_platform'].unique()
```

Out[19]:

```
array(['direct online', 'others', 'logtrip', 'tripster', 'makeyourtrip',
      'journey', 'direct offline'], dtype=object)
```

In [215...

```
# cheking no of bookings by each platform
df_bookings['booking_platform'].value_counts()
```

Out[215...

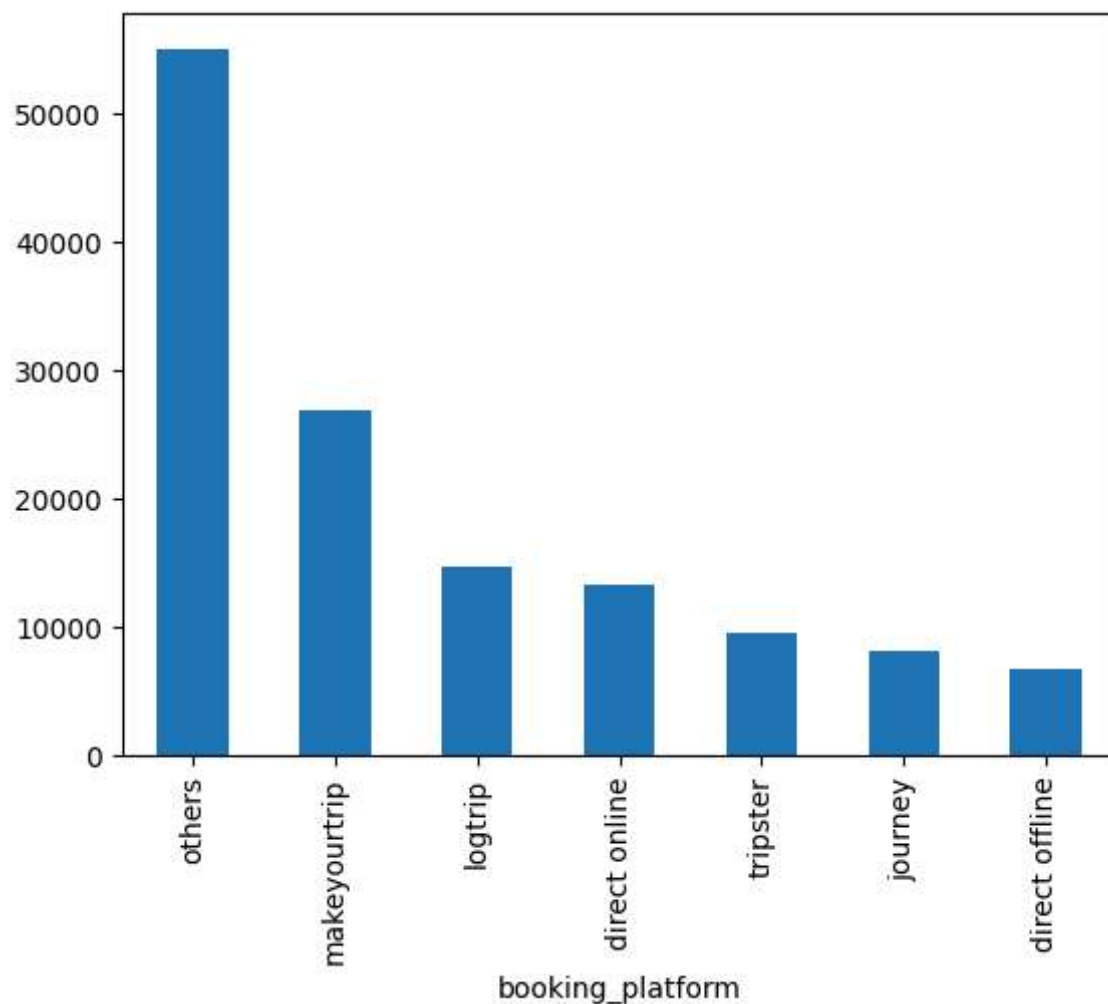
```
booking_platform
others          55066
makeyourtrip    26898
logtrip         14756
direct online   13379
tripster        9630
journey         8106
direct offline  6755
Name: count, dtype: int64
```

In [216...

```
#plotting a bar chart for no of bookings by each platform
df_bookings['booking_platform'].value_counts().plot(kind='bar')
```

Out[216...

```
<Axes: xlabel='booking_platform'>
```



loading other files

```
In [22]: # loading other files
df_date=pd.read_csv(r"D:\Python_study\Python_Practice\source-code\python_projects\3_project_hotels\date.csv")
df_hotels=pd.read_csv(r"D:\Python_study\Python_Practice\source-code\python_projects\3_project_hotels\hotels.csv")
df_rooms=pd.read_csv(r"D:\Python_study\Python_Practice\source-code\python_projects\3_project_hotels\rooms.csv")
df_agg_booking=pd.read_csv(r"D:\Python_study\Python_Practice\source-code\python_projects\3_project_hotels\agg_booking.csv")
```

```
In [23]: df_hotels.head(5)
```

```
Out[23]:
```

	property_id	property_name	category	city
0	16558	Atliq Grands	Luxury	Delhi
1	16559	Atliq Exotica	Luxury	Mumbai
2	16560	Atliq City	Business	Delhi
3	16561	Atliq Blu	Luxury	Delhi
4	16562	Atliq Bay	Luxury	Delhi

checking numbers of category

```
In [24]: df_hotels['category'].value_counts()
```

```
Out[24]: category
Luxury      16
Business     9
Name: count, dtype: int64
```

checking property_name with their count


```
In [25]: df_hotels['property_name'].value_counts()
```

```
Out[25]: property_name
Atliq Grands      4
Atliq Exotica     4
Atliq City        4
Atliq Blu         4
Atliq Bay         4
Atliq Palace      4
Atliq Seasons     1
Name: count, dtype: int64
```

checking no of hotels per city in ascending order

```
In [26]: df_hotels['city'].value_counts().sort_values()
```

```
Out[26]: city
Delhi        5
Hyderabad    6
Bangalore    6
Mumbai       8
Name: count, dtype: int64
```

checking missing value,doing this for others file as well

```
In [27]: # checking for missing value,& check for other file as well
df_hotels.isna().sum()
```

```
Out[27]: property_id      0
property_name      0
category           0
city              0
dtype: int64
```

Find out unique property ids in aggregate bookings dataset

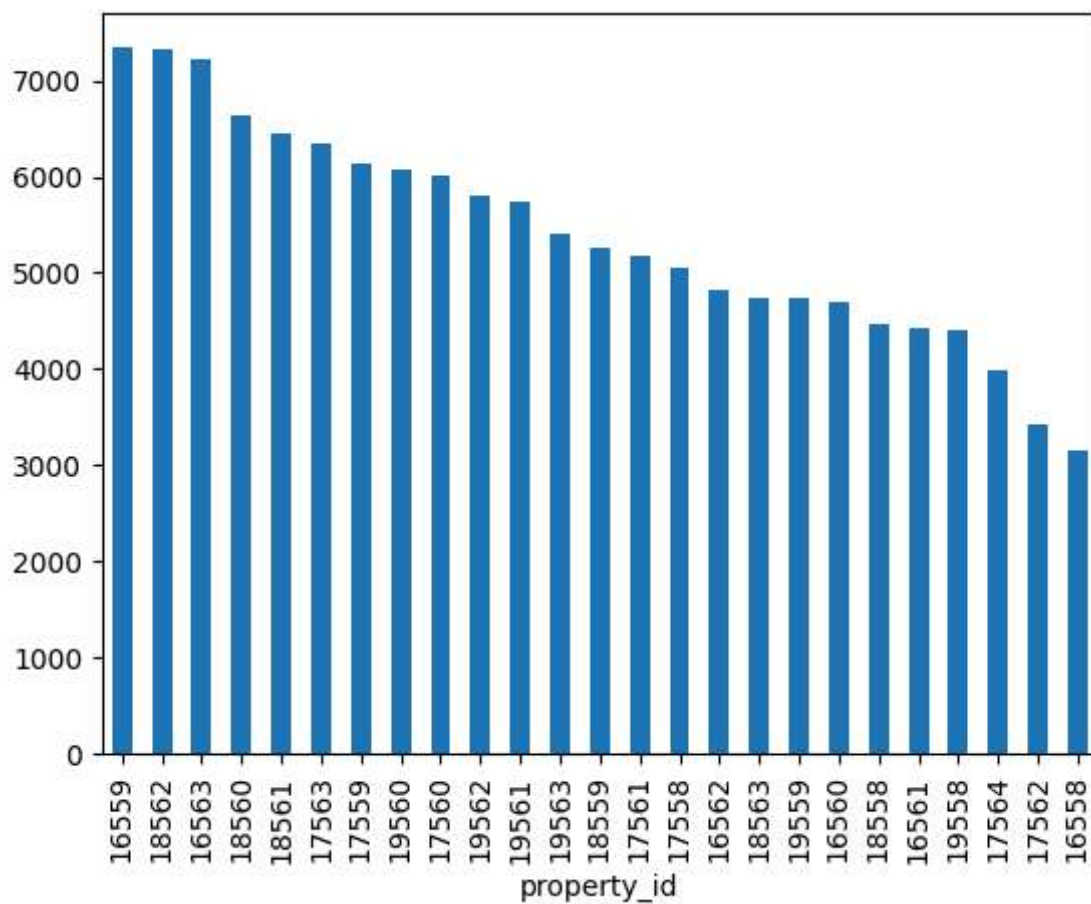
```
In [217... df_agg_booking['property_id'].unique()
```

```
Out[217... array([16559, 19562, 19563, 17558, 16558, 17560, 19558, 19560, 17561,
        16560, 16561, 16562, 16563, 17559, 17562, 17563, 18558, 18559,
        18561, 18562, 18563, 19559, 19561, 17564, 18560])
```

Find out total bookings per property_id

```
In [297... total_bookings_per_property_id=df_agg_booking.groupby('property_id')['successful_bookings'].sum()
total_bookings_per_property_id.sort_values(ascending=False).plot(kind='bar')
```

```
Out[297... <Axes: xlabel='property_id'>
```



```
In [30]: df_agg_booking.head(2)
```

```
Out[30]:
```

	property_id	check_in_date	room_category	successful_bookings	capacity
0	16559	1-May-22	RT1	25	30.0
1	19562	1-May-22	RT1	28	30.0

Days where booking was greater than capacity

```
In [219]: df_agg_booking['check_in_date'][(df_agg_booking['successful_bookings']>df_agg_booking['capacity'])]
```

```
Out[219]:
```

3	1-May-22
12	1-May-22
4136	11-Jun-22
6209	2-Jul-22
8522	25-Jul-22
9194	31-Jul-22

Name: check_in_date, dtype: object

Properties that have highest capacity

```
In [220]: df_agg_booking['property_id'][(df_agg_booking['capacity']==df_agg_booking['capacity'].max())]
```

```
Out[220]:
```

27	17558
128	17558
229	17558
328	17558
428	17558
...	
8728	17558
8828	17558
8928	17558
9028	17558
9128	17558

Name: property_id, Length: 92, dtype: int64

Data Cleaning

In [221... df_bookings.describe()

Out[221...

	property_id	no_guests	ratings_given	revenue_generated	revenue_realized
count	134590.000000	134587.000000	56683.000000	1.345900e+05	134590.000000
mean	18061.113493	2.036170	3.619004	1.537805e+04	12696.123256
std	1093.055847	1.034885	1.235009	9.303604e+04	6928.108124
min	16558.000000	-17.000000	1.000000	6.500000e+03	2600.000000
25%	17558.000000	1.000000	3.000000	9.900000e+03	7600.000000
50%	17564.000000	2.000000	4.000000	1.350000e+04	11700.000000
75%	18563.000000	2.000000	5.000000	1.800000e+04	15300.000000
max	19563.000000	6.000000	5.000000	2.856000e+07	45220.000000

Rows those have negative value in no_guests column

In [222...

```
df_bookings[(df_bookings['no_guests']<=0)]  
## remove from df_bookings where no_guests has negative value,& saving it to df_bookings  
df_bookings=df_bookings[(df_bookings['no_guests']>0)]  
df_bookings.head(5)
```

Out[222...

	booking_id	property_id	booking_date	check_in_date	checkout_date	no_guests	room_cate
1	May012216558RT12	16558	30-04-22	1/5/2022	2/5/2022	2.0	
2	May012216558RT13	16558	28-04-22	1/5/2022	4/5/2022	2.0	
4	May012216558RT15	16558	27-04-22	1/5/2022	2/5/2022	4.0	
5	May012216558RT16	16558	1/5/2022	1/5/2022	3/5/2022	2.0	
6	May012216558RT17	16558	28-04-22	1/5/2022	6/5/2022	2.0	

Outliers finding

In [186...

```
## data type revenue_generated column  
df_bookings['revenue_generated'].max(),df_bookings['revenue_generated'].min()
```

Out[186...

```
(np.int64(28560000), np.int64(6500))
```

In [187...

```
# calculating average & standered daviation of revenue_generated column  
mean,std=df_bookings['revenue_generated'].mean(),df_bookings['revenue_generated'].std()  
print(mean),print(std)
```

```
15377.943639470946  
93039.46673432393
```

Out[187...

```
(None, None)
```

In [188...

```
# calculating higher limit= (mean + 3*std) & Lower limit=(mean - 3*std)  
higher_limit=mean+3*std  
lower_limit=mean-3*std  
higher_limit,lower_limit
```

Out[188... (np.float64(294496.34384244273), np.float64(-263740.4565635008))

```
In [223... #now check out outliers based on lower_limit
# no value fould,so data is good based on lower_limit
df_bookings[(df_bookings['revenue_generated']<0)]
```

Out[223... **booking_id** **property_id** **booking_date** **check_in_date** **checkout_date** **no_guests** **room_category** **bo**



```
In [224... #now check out outliers based on higher_limit
df_bookings[(df_bookings['revenue_generated']>higher_limit)]
```

Out[224... **booking_id** **property_id** **booking_date** **check_in_date** **checkout_date** **no_guests** **roo**

2	May012216558RT13	16558	28-04-22	1/5/2022	4/5/2022	2.0
113	May012216559RT32	16559	29-04-22	1/5/2022	2/5/2022	6.0
317	May012216562RT22	16562	28-04-22	1/5/2022	4/5/2022	2.0
564	May012217559RT118	17559	26-04-22	1/5/2022	2/5/2022	2.0
129178	Jul282216562RT26	16562	21-07-22	28-07-22	29-07-22	2.0



```
In [225... # now we are going to remove those rows those are more than the higher limit,that means we wl
# will save that result to df_bookings,so now df_bookings is clean based on outliers
df_bookings=df_bookings[(df_bookings['revenue_generated']<higher_limit)]
df_bookings.shape
```

Out[225... (134573, 12)

```
In [226... #checking quick statistics for revenue_realized column
df_bookings['revenue_realized'].describe()
```

Out[226... count 134573.000000
mean 12695.983585
std 6927.791692
min 2600.000000
25% 7600.000000
50% 11700.000000
75% 15300.000000
max 45220.000000
Name: revenue_realized, dtype: float64

```
In [227... #checking 3 std check for revenue_realized column
higher_limit=df_bookings['revenue_realized'].mean()+3*df_bookings['revenue_realized'].std()
higher_limit
# the max value is 45220 & higher_limit is 33479,it looks like a outlier ,but we want to deep
df_bookings[(df_bookings['revenue_realized']>higher_limit)]
# we found all the filtered columns has room_category is RT4
```

Out[227...

	booking_id	property_id	booking_date	check_in_date	checkout_date	no_guests	roo
	139	May012216559RT41	16559	27-04-22	1/5/2022	7/5/2022	4.0
	141	May012216559RT43	16559	1/5/2022	1/5/2022	2/5/2022	6.0
	145	May012216559RT47	16559	28-04-22	1/5/2022	3/5/2022	3.0
	151	May012216559RT413	16559	24-04-22	1/5/2022	7/5/2022	5.0
	224	May012216560RT45	16560	30-04-22	1/5/2022	3/5/2022	5.0
	...	...	...	...	...	...	...
	134330	Jul312219560RT49	19560	31-07-22	31-07-22	2/8/2022	6.0
	134333	Jul312219560RT412	19560	31-07-22	31-07-22	1/8/2022	6.0
	134469	Jul312219562RT45	19562	28-07-22	31-07-22	1/8/2022	6.0
	134476	Jul312219562RT412	19562	25-07-22	31-07-22	6/8/2022	5.0
	134583	Jul312217564RT42	17564	31-07-22	31-07-22	1/8/2022	4.0

1299 rows × 12 columns



In [228...

```
## to find out the outliers we need to find the std & mean of revenue_realized for RT4 rooms
df_bookings[(df_bookings['room_category']=='RT4')].revenue_realized.describe()
#df_bookings['room_category']=='RT4'
```

Out[228...

```
count    16071.000000
mean     23439.308444
std       9048.599076
min       7600.000000
25%      19000.000000
50%      26600.000000
75%      32300.000000
max      45220.000000
Name: revenue_realized, dtype: float64
```

In [229...

```
# calculating the higher_limit, lower_limit
higher_limit=23439.30+3*9048.59
lower_limit=23439.30-3*9048.59
higher_limit, lower_limit
# our max value is 45220 which is < higher_limit so there has no outlier in the data .
```

Out[229...

```
(50585.07, -3706.470000000001)
```

In aggregate bookings find columns that have null values. Fill these null values with whatever you think is the appropriate substitute (possible ways is to use mean or median)

In [44]:

```
df_agg_booking.isnull().sum()
```

Out[44]:

```
property_id      0
check_in_date    0
room_category    0
successful_bookings  0
capacity         2
dtype: int64
```

In [230...

```
#columns that have null values.
df_agg_booking[(df_agg_booking['capacity'].isnull())]
```

```

Out[230]:
    property_id  check_in_date  room_category  successful_bookings  capacity  occ_pct

In [46]: #we can replace 2 null values with the median
df_agg_booking['capacity'].median()

Out[46]: np.float64(25.0)

In [47]: df_agg_booking['capacity']=df_agg_booking['capacity'].fillna(df_agg_booking['capacity'].median())

In [48]: df_agg_booking.isnull().sum()

Out[48]:
property_id      0
check_in_date    0
room_category    0
successful_bookings  0
capacity         0
dtype: int64

```

In aggregate bookings find out records that have successful_bookings value greater than capacity. Filter those records

```
In [49]: df_agg_booking[(df_agg_booking['successful_bookings']>df_agg_booking['capacity'])]
```

```

Out[49]:
   property_id  check_in_date  room_category  successful_bookings  capacity
3           17558      1-May-22           RT1                    30        19.0
12          16563      1-May-22           RT1                   100        41.0
4136         19558     11-Jun-22           RT2                    50        39.0
6209         19560      2-Jul-22           RT1                   123        26.0
8522         19559     25-Jul-22           RT1                    35        24.0
9194         18563     31-Jul-22           RT4                    20        18.0

```

Data Transformation

Adding a new column (occ_pct) Occupancy Percentage in df_agg_booking table, where Occupancy=(successful_bookings/capacity)

```
In [50]: df_agg_booking['occ_pct']=df_agg_booking['successful_bookings']/df_agg_booking['capacity']
df_agg_booking.head(2)
```

```

Out[50]:
   property_id  check_in_date  room_category  successful_bookings  capacity  occ_pct
0           16559      1-May-22           RT1                    25        30.0  0.833333
1           19562      1-May-22           RT1                    28        30.0  0.933333

```

```

In [51]: # converting percent & rounding occ_pct column.

df_agg_booking['occ_pct']=df_agg_booking['occ_pct'].apply(lambda x:round(x *100,2))
df_agg_booking.head(2)

```

Out[51]:

	property_id	check_in_date	room_category	successful_bookings	capacity	occ_pct
0	16559	1-May-22	RT1	25	30.0	83.33
1	19562	1-May-22	RT1	28	30.0	93.33

Insights Generation

1. What is an average occupancy rate in each of the room categories?

In [52]: df_rooms

Out[52]:

	room_id	room_class
0	RT1	Standard
1	RT2	Elite
2	RT3	Premium
3	RT4	Presidential

In [53]: df_agg_booking.head(2)

Out[53]:

	property_id	check_in_date	room_category	successful_bookings	capacity	occ_pct
0	16559	1-May-22	RT1	25	30.0	83.33
1	19562	1-May-22	RT1	28	30.0	93.33

In [54]: *#joining df_agg_booking with df_rooms to get room_class*
df=pd.merge(df_agg_booking,df_rooms,left_on='room_category',right_on='room_id')

dropping column room_id from df as room_category & room_id both are same
df.drop('room_id',axis=1,inplace=True)
df.head(3)

Out[54]:

	property_id	check_in_date	room_category	successful_bookings	capacity	occ_pct	room_class
0	16559	1-May-22	RT1	25	30.0	83.33	Standard
1	19562	1-May-22	RT1	28	30.0	93.33	Standard
2	19563	1-May-22	RT1	23	30.0	76.67	Standard

In [55]: *# Finding average occupancy rate in each of the room categories*
df.groupby('room_class')['occ_pct'].mean().round(2)

Out[55]:

room_class	
Elite	58.04
Premium	58.03
Presidential	59.30
Standard	58.23
Name: occ_pct, dtype: float64	

2. Print average occupancy rate per city

In [56]: df_hotels.head(2)

Out[56]:

	property_id	property_name	category	city
0	16558	Atliq Grands	Luxury	Delhi
1	16559	Atliq Exotica	Luxury	Mumbai

In [57]: df.head(2)

Out[57]:

	property_id	check_in_date	room_category	successful_bookings	capacity	occ_pct	room_class
0	16559	1-May-22	RT1	25	30.0	83.33	Standard
1	19562	1-May-22	RT1	28	30.0	93.33	Standard

In [58]: # joining df with df_hotels to get city ,primary key property_id
df=pd.merge(df,df_hotels,on='property_id',how='left')

Calculating average occupancy rate per city
df.groupby('city')['occ_pct'].mean().round(2)

Out[58]:

city	
Bangalore	56.59
Delhi	61.61
Hyderabad	58.14
Mumbai	57.94
Name: occ_pct, dtype: float64	

3. When was the occupancy better? Weekday or Weekend?

In [59]: df_date.head(3)

Out[59]:

	date	mmm yy	week no	day_type
0	01-May-22	May 22	W 19	weekend
1	02-May-22	May 22	W 19	weekeday
2	03-May-22	May 22	W 19	weekeday

In [60]: df.head(2)

Out[60]:

	property_id	check_in_date	room_category	successful_bookings	capacity	occ_pct	room_class	pro
0	16559	1-May-22	RT1	25	30.0	83.33	Standard	
1	19562	1-May-22	RT1	28	30.0	93.33	Standard	



In [62]: # Joining df with df_date to get day_type,primary key 'check_in_date' for df & 'date' for df_c
df=pd.merge(df,df_date,left_on='check_in_date',right_on='date')
df.drop(['date_x','mmm yy_x','week no_x','day_type_x'],axis=1,inplace=True)

df.head(2)

Out[62]:

	property_id	check_in_date	room_category	successful_bookings	capacity	occ_pct	room_class	pro
0	19563	10-May-22	RT3	15	29.0	51.72	Premium	
1	18560	10-May-22	RT1	19	30.0	63.33	Standard	

In [63]: *# Finding occupancy based on Weekday or Weekend*
df.groupby('day_type')['occ_pct'].mean().round(2)

Out[63]: day_type
weekday 50.90
weekend 72.39
Name: occ_pct, dtype: float64

4: In the month of June, what is the occupancy for different cities

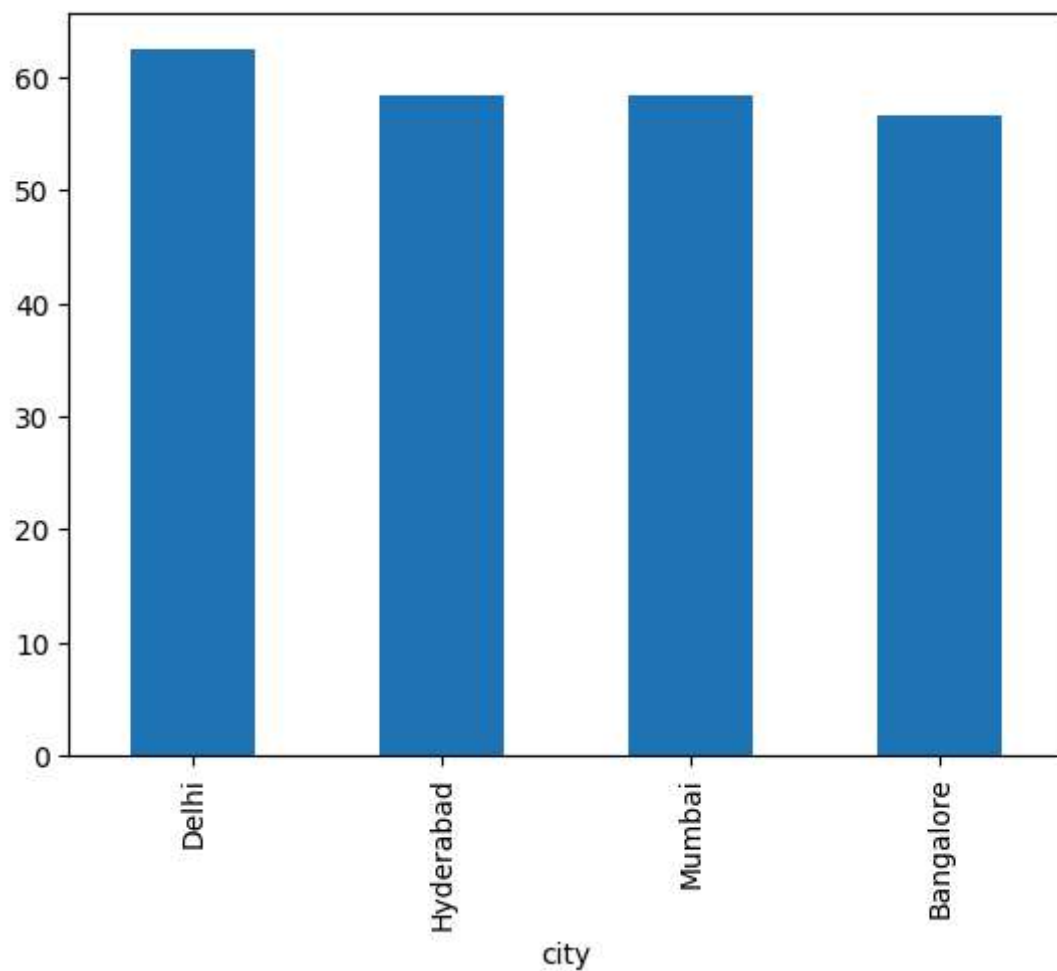
In [64]: df.head(2)

Out[64]:

	property_id	check_in_date	room_category	successful_bookings	capacity	occ_pct	room_class	pro
0	19563	10-May-22	RT3	15	29.0	51.72	Premium	
1	18560	10-May-22	RT1	19	30.0	63.33	Standard	

In [73]: df_jun=df[(df['mmm yy']=='Jun 22')]
df_jun.groupby('city')['occ_pct'].mean().sort_values(ascending=False).plot(kind='bar')

Out[73]: <Axes: xlabel='city'>



5: We got new data for the month of august. Append that to existing data

In [74]: `df_august=pd.read_csv(r"D:\Python_study\Python_Practice\source-code\python_projects\3_project\df_august`

Out[74]:

	property_id	property_name	category	city	room_category	room_class	check_in_date	mmm yy
0	16559	Atliq Exotica	Luxury	Mumbai	RT1	Standard	01-Aug-22	Aug-22
1	19562	Atliq Bay	Luxury	Bangalore	RT1	Standard	01-Aug-22	Aug-22
2	19563	Atliq Palace	Business	Bangalore	RT1	Standard	01-Aug-22	Aug-22
3	19558	Atliq Grands	Luxury	Bangalore	RT1	Standard	01-Aug-22	Aug-22
4	19560	Atliq City	Business	Bangalore	RT1	Standard	01-Aug-22	Aug-22
5	17561	Atliq Blu	Luxury	Mumbai	RT1	Standard	01-Aug-22	Aug-22
6	17564	Atliq Seasons	Business	Mumbai	RT1	Standard	01-Aug-22	Aug-22

In [75]: `#checking the columns in df_august
df_august.columns`

```
Out[75]: Index(['property_id', 'property_name', 'category', 'city', 'room_category',
              'room_class', 'check_in_date', 'mmm yy', 'week no', 'day_type',
              'successful_bookings', 'capacity', 'occ%'],
              dtype='object')
```

```
In [76]: #checking the columns in df
df.columns
```

```
Out[76]: Index(['property_id', 'check_in_date', 'room_category', 'successful_bookings',
              'capacity', 'occ_pct', 'room_class', 'property_name', 'category',
              'city', 'date', 'mmm yy', 'week no', 'day_type'],
              dtype='object')
```

```
In [78]: #checking no of rows & columns in df_august
df_august.shape
```

```
Out[78]: (7, 13)
```

```
In [79]: #checking no of rows & columns in df
df.shape
```

```
Out[79]: (2100, 14)
```

```
In [298... # Concatinating df_august with df
latest_df=pd.concat([df,df_august],ignore_index=True,axis=0)
latest_df.tail(4)
```

Out[298...

	booking_id	property_id	booking_date	check_in_date	checkout_date	no_guests	room_category
	134576	NaN	19558	NaT	01-Aug-22	NaN	NaN
	134577	NaN	19560	NaT	01-Aug-22	NaN	NaN
	134578	NaN	17561	NaT	01-Aug-22	NaN	NaN
	134579	NaN	17564	NaT	01-Aug-22	NaN	NaN

4 rows × 22 columns

```
In [83]: latest_df.shape
```

```
Out[83]: (2107, 15)
```

6. Print revenue realized per city

```
In [ ]: df_date
df_hotels
df_rooms
df_agg_booking
```

```
In [231... df_bookings.head(2)
```

Out[231...	booking_id	property_id	booking_date	check_in_date	checkout_date	no_guests	room_cate
	1	May012216558RT12	16558	30-04-22	1/5/2022	2/5/2022	2.0
	4	May012216558RT15	16558	27-04-22	1/5/2022	2/5/2022	4.0

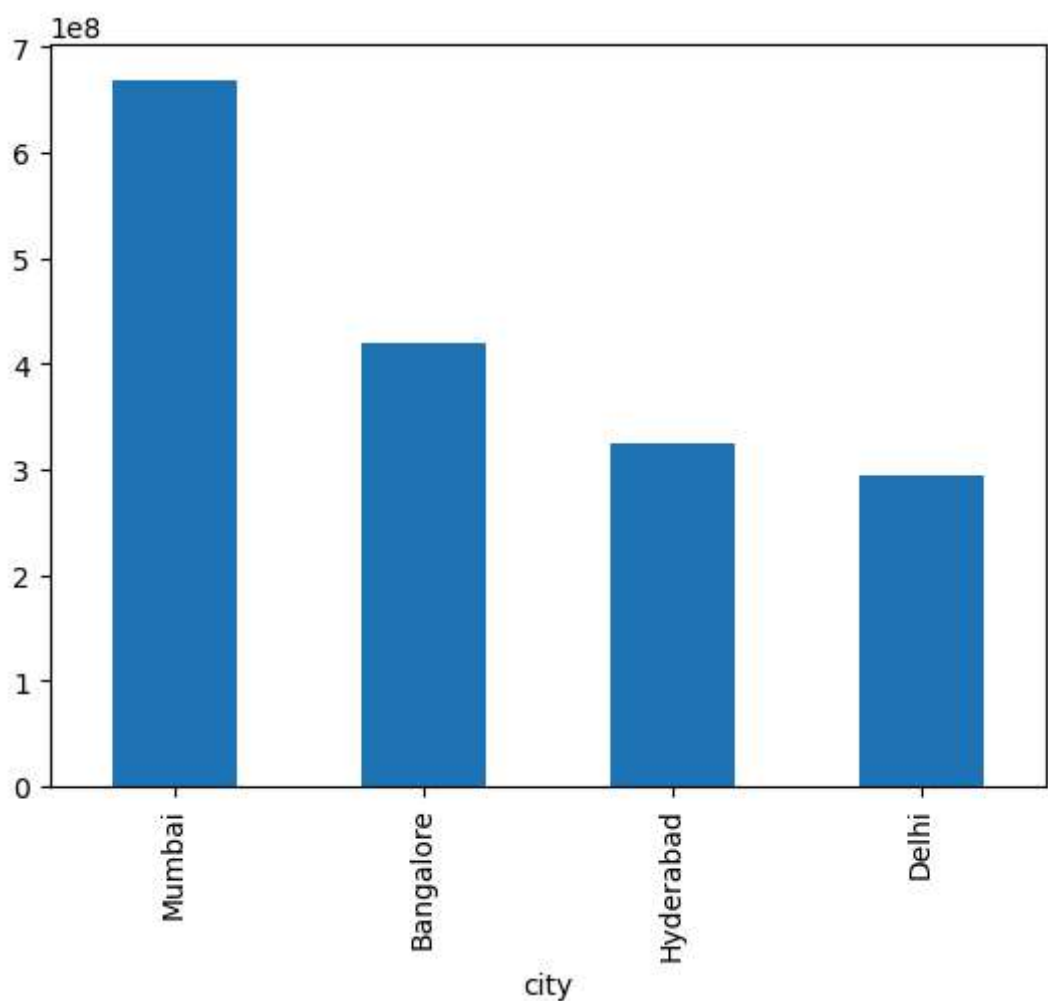
```
df_hotels.head(2)
```

Out[232...

	property_id	property_name	category	city
0	16558	Atliq Grands	Luxury	Delhi
1	16559	Atliq Exotica	Luxury	Mumbai

```
booking_all=pd.merge(df_bookings,df_hotels,on='property_id',how='left')
booking_all.groupby('city')['revenue_realized'].sum().sort_values(ascending=False).plot(kind=
```

```
<Axes: xlabel='city'>
```



7. Print month by month revenue

```
df_date.head(3)
```

Out[299...

	date	mmm yy	week no	day_type
0	2022-05-01	May 22	W 19	weekend
1	2022-05-02	May 22	W 19	weekday
2	2022-05-03	May 22	W 19	weekday

```
In [121... df_date.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 92 entries, 0 to 91
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   date        92 non-null    object
1   mmm yy      92 non-null    object
2   week no     92 non-null    object
3   day_type    92 non-null    object
dtypes: object(4)
memory usage: 3.0+ KB

In [153... # converting date column to datetime format
df_date['date']=pd.to_datetime(df_date['date'])
df_date.head(2)
```

```
Out[153...      date  mmm yy  week no  day_type
0  2022-05-01   May 22    W 19   weekend
1  2022-05-02   May 22    W 19  weekday
```

```
In [252... df_date.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 92 entries, 0 to 91
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   date        92 non-null    datetime64[ns]
1   mmm yy      92 non-null    object
2   week no     92 non-null    object
3   day_type    92 non-null    object
dtypes: datetime64[ns](1), object(3)
memory usage: 3.0+ KB
```

```
In [236... df_bookings.head(2)
```

```
Out[236...      booking_id  property_id  booking_date  check_in_date  checkout_date  no_guests  room_cate
1  May012216558RT12      16558    30-04-22    1/5/2022    2/5/2022        2.0
4  May012216558RT15      16558    27-04-22    1/5/2022    2/5/2022        4.0
```

```
In [237... df_bookings['booking_date']

Out[237... 1      30-04-22
4      27-04-22
5      1/5/2022
6      28-04-22
8      26-04-22
...
134586 30-07-22
134587 29-07-22
134589 30-07-22
134590 29-07-22
134591 31-07-22
Name: booking_date, Length: 134573, dtype: object

In [267... df_bookings['booking_date']=pd.to_datetime(df_bookings['booking_date'])
df_bookings.info()
```

```

<class 'pandas.core.frame.DataFrame'>
Index: 134573 entries, 1 to 134591
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   booking_id            134573 non-null object
1   property_id           134573 non-null int64
2   booking_date          134573 non-null datetime64[ns]
3   check_in_date         134573 non-null object
4   checkout_date         134573 non-null object
5   no_guests             134573 non-null float64
6   room_category         134573 non-null object
7   booking_platform      134573 non-null object
8   ratings_given         56676 non-null float64
9   booking_status        134573 non-null object
10  revenue_generated     134573 non-null int64
11  revenue_realized      134573 non-null int64
dtypes: datetime64[ns](1), float64(2), int64(3), object(6)
memory usage: 13.3+ MB

```

```

In [274... # joining df_bookings['booking_date'] with df_date['date']
df_total=pd.merge(df_bookings,df_date,left_on='booking_date',right_on='date')
df_total.head(3)

```

```

Out[274...

```

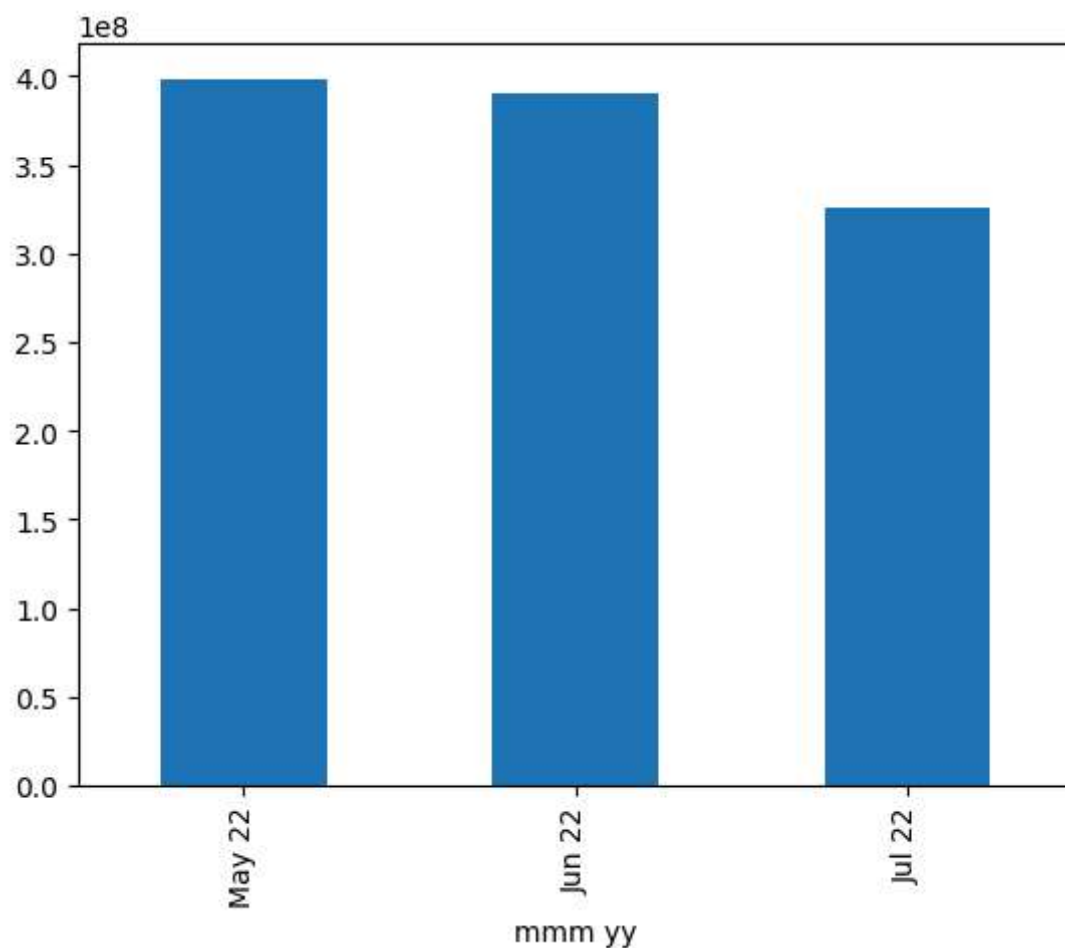
	booking_id	property_id	booking_date	check_in_date	checkout_date	no_guests	room_cat
0	May012216558RT21	16558	2022-07-04	1/5/2022	6/5/2022	2.0	
1	May012216559RT320	16559	2022-07-04	1/5/2022	2/5/2022	3.0	
2	May012216560RT22	16560	2022-07-04	1/5/2022	2/5/2022	2.0	

```

In [277... #Grouping mmm yy based on revenue_realized
df_total.groupby('mmm yy')['revenue_realized'].sum().sort_values(ascending=False).plot(kind='bar')

Out[277... <Axes: xlabel='mmm yy'>

```



Print revenue realized per hotel type

```
In [ ]: df_date
df_hotels
df_rooms
df_agg_booking
```

```
In [284...] # merging df_bookings with df_hotels on property_id
df=pd.merge(df_bookings,df_hotels,on='property_id',how='left')
df.head(2)
```

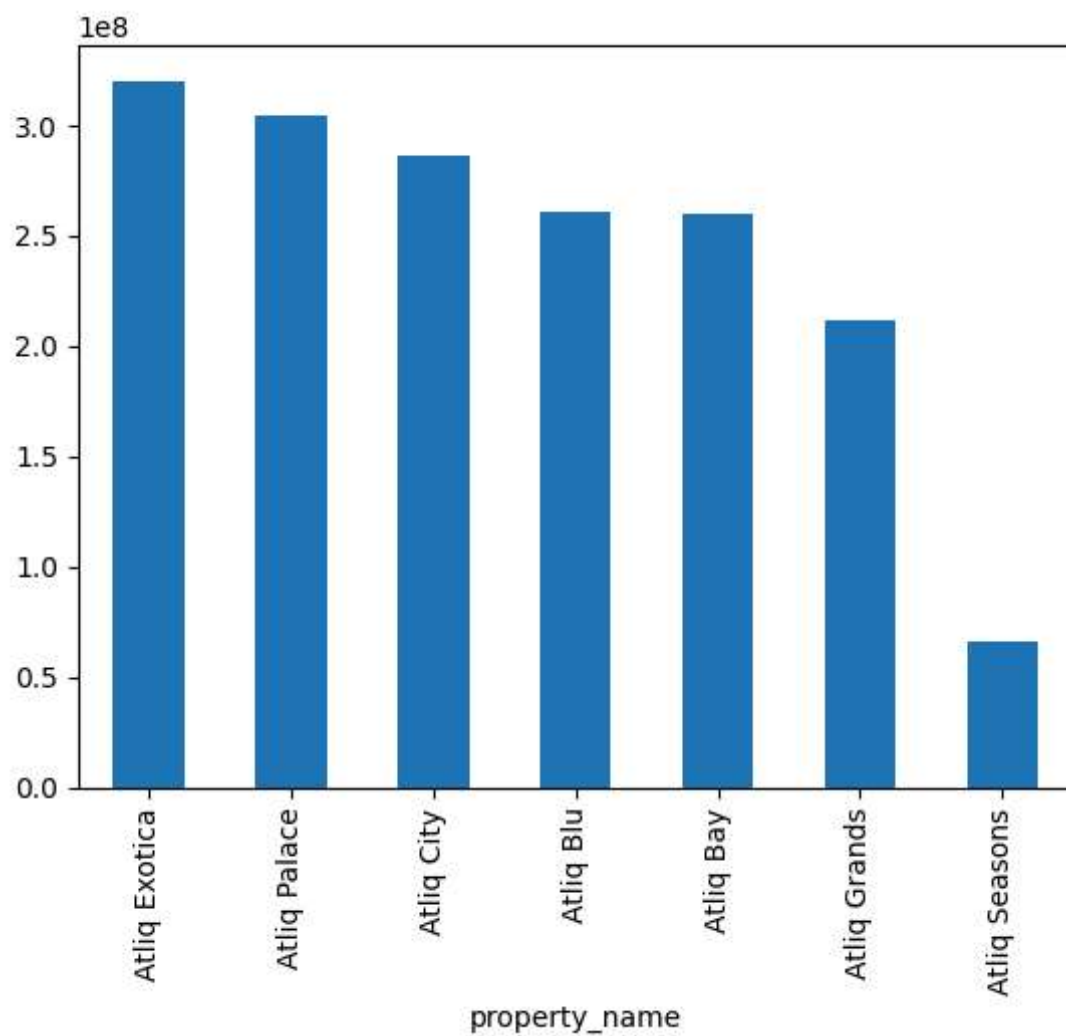
```
Out[284...]
```

	booking_id	property_id	booking_date	check_in_date	checkout_date	no_guests	room_cate
0	May012216558RT12	16558	2022-04-30	1/5/2022	2/5/2022	2.0	
1	May012216558RT15	16558	2022-04-27	1/5/2022	2/5/2022	4.0	



```
In [287...] # grouping property_name based on revenue_realized
df.groupby('property_name')['revenue_realized'].sum().sort_values(ascending=False).plot(kind=
```

```
Out[287...] <Axes: xlabel='property_name'>
```



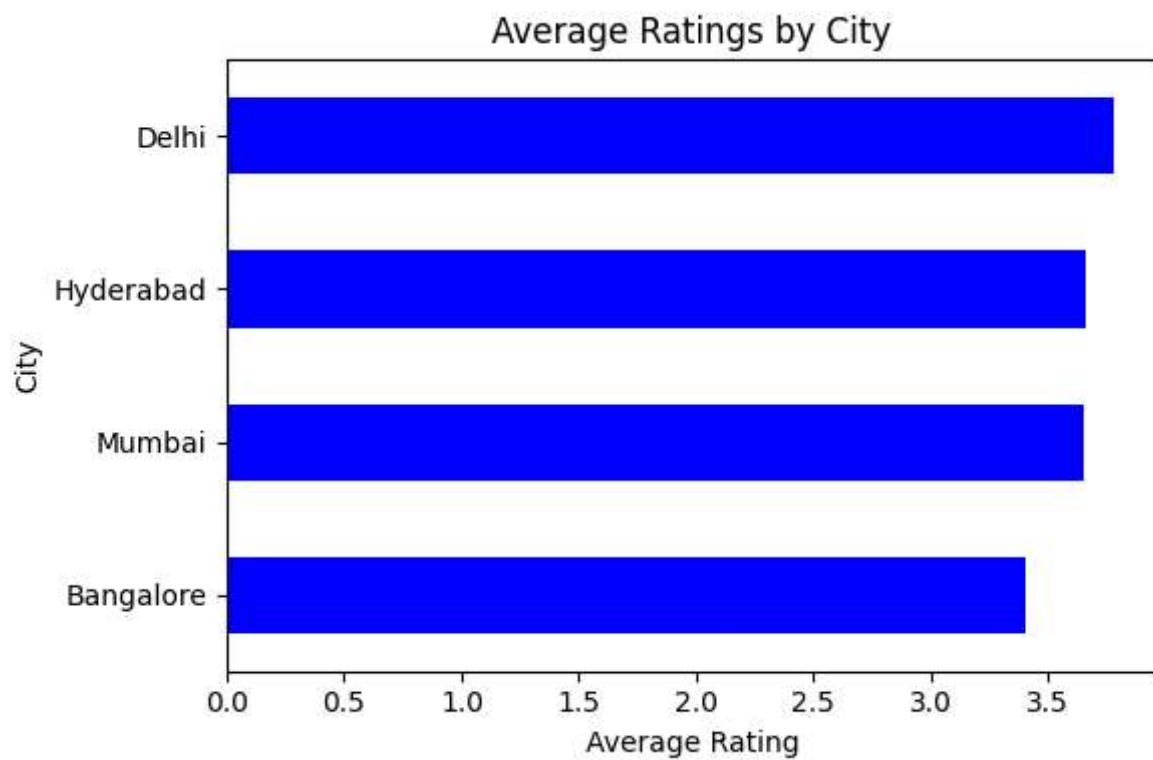
Print average rating per city

In [332...

```
import matplotlib.pyplot as plt

df.groupby('city')['ratings_given'].mean().sort_values(ascending=True).plot(
    kind='barh',
    figsize=(6, 4),
    color='blue',
    title='Average Ratings by City'
)

plt.xlabel('Average Rating')
plt.ylabel('City')
plt.tight_layout()
plt.show()
```

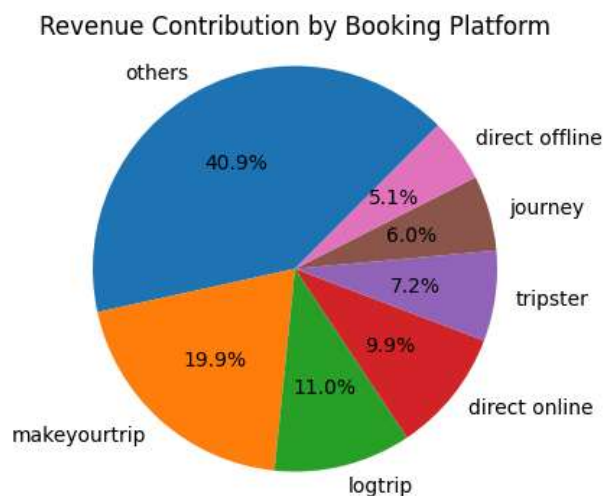
Print a pie chart of revenue realized per booking platform

In [319...

```
import matplotlib.pyplot as plt

# Grouping and sorting revenue by booking platform
df.groupby('booking_platform')['revenue_realized'].sum().sort_values(ascending=False).plot(
    kind='pie',
    autopct='%1.1f%%',
    startangle=45,
    figsize=(12,4),
    title='Revenue Contribution by Booking Platform'
)

plt.ylabel('')          # Optional: removes y-axis label
plt.axis('equal')       # Keeps the pie circular
plt.show()
```



In []: